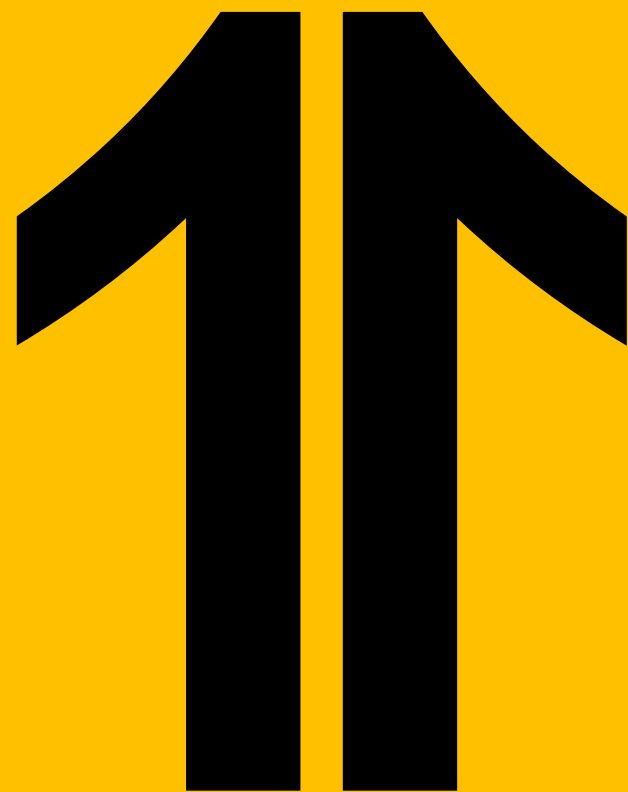




UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**



EVOLUCIÓN Y CONFIGURACIÓN DE SOFTWARE

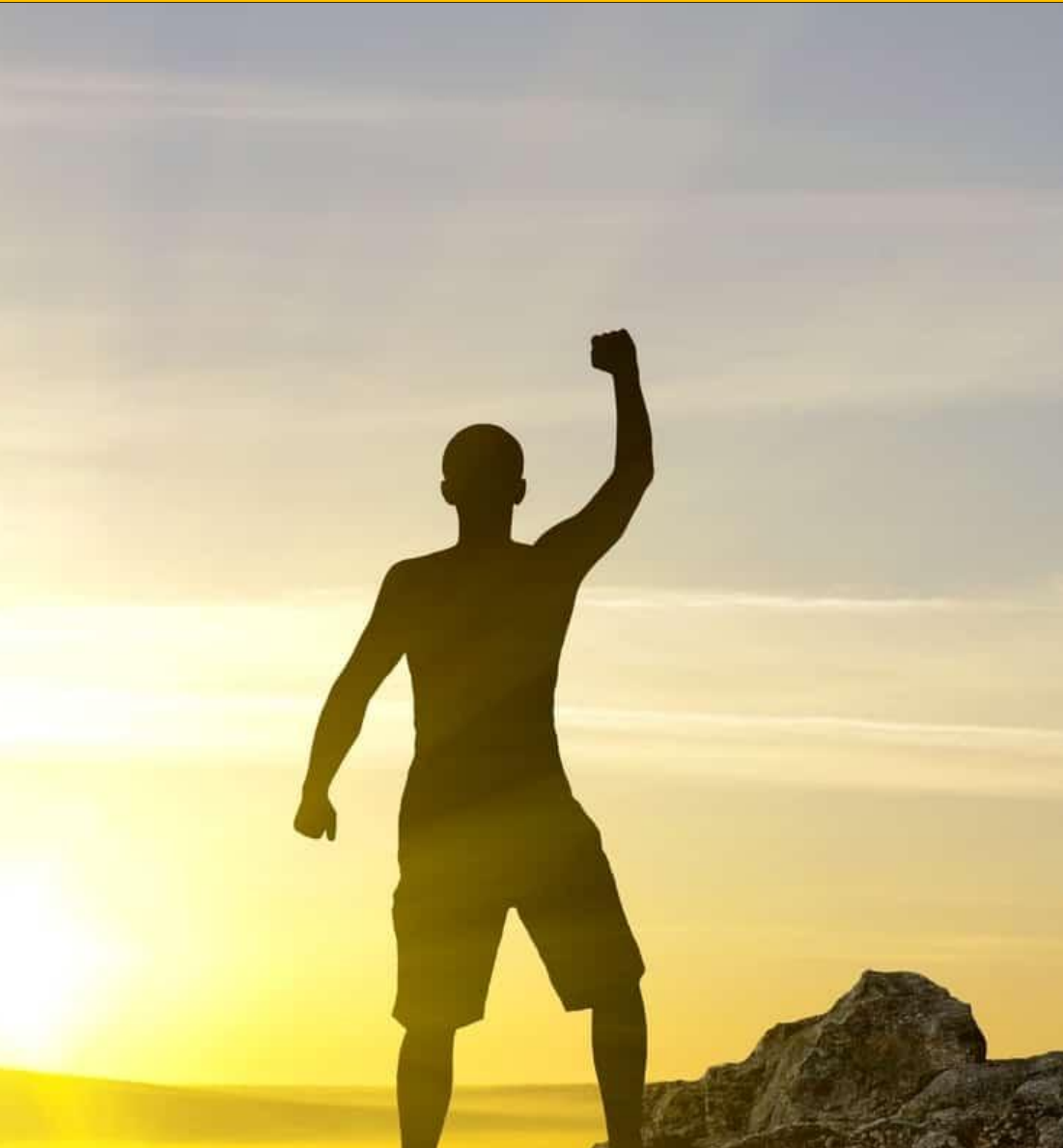
Semana 4

Ing. Edgard Vilchez Valdez.

edgard.vilchez@upn.pe



Logros de la sesión



Al finalizar la sesión, los estudiantes habrán comprendido y aplica técnicas de refactorización utilizando el Principio de Única Responsabilidad y DRY para mejorar la estructura del código, reducir la duplicidad y aumentar la mantenibilidad del software.

Índice de la sesión



Índice	Tema
1.1	Refactorización y el Principio de Única Responsabilidad y DRY.



Trabajo Grupal: Semana 4



Estimados estudiantes,
Para el próximo trabajo grupal, cada grupo deberá preparar una exposición sobre uno de los siguientes temas. El objetivo es que cada grupo profundice en su tema asignado, presente los conceptos teóricos y prácticos, y los ejemplifique con código funcional.





Objetivo del trabajo

El presente trabajo tiene como finalidad que el estudiante investigue, analice y aplique los conceptos de refactorización, Principio de Única Responsabilidad (Single Responsibility Principle – SRP) y DRY (Don't Repeat Yourself), relacionándolos con la mejora de la mantenibilidad, legibilidad y evolución del software.



 Investigar los siguientes temas:

Refactorización

- Definición de refactorización.
- Objetivos de la refactorización.
- Beneficios de refactorizar un sistema.
- Riesgos de refactorizar sin análisis previo o sin pruebas.
- Importancia de la refactorización en sistemas heredados o de larga evolución.



 Investigar los siguientes temas:

Principio de Única Responsabilidad (SRP)

- Definición del principio.
- Qué significa que una clase, módulo o componente tenga una sola responsabilidad.
- Problemas que aparecen cuando una clase tiene múltiples responsabilidades.
- Ventajas de aplicar SRP.
- Ejemplos de violación y cumplimiento del principio.

Trabajo Grupal: Semana 4



 Investigar los siguientes temas:

Principio DRY

- Definición del principio DRY.
- Qué se entiende por duplicación de lógica, código, reglas o procesos.
- Problemas que genera la duplicidad.
- Beneficios de aplicar DRY.
- Ejemplos de código repetido y su mejora.

Relación entre refactorización, SRP y DRY

- Cómo se complementan estos conceptos.
- Cómo contribuyen a la mantenibilidad del software.
- Cómo ayudan en la evolución del sistema.
- Su impacto en pruebas, correcciones y futuras ampliaciones.



Desarrollo Práctico

A partir del avance actual de su proyecto final, el estudiante o equipo debe identificar una parte del sistema donde sea posible analizar problemas de diseño o mantenibilidad, tales como:

- clases o módulos que realizan demasiadas funciones;
- métodos extensos;
- repetición de bloques de código;
- validaciones repetidas;
- lógica mezclada con acceso a datos, interfaz o reglas de negocio;
- estructuras difíciles de mantener o ampliar.

Diagnóstico del problema

- Identificar y explicar los problemas encontrados en esa parte del proyecto:
- dónde se observa falta de SRP;
- dónde existe duplicidad que incumple DRY;
- qué elementos requieren refactorización;
- por qué esa situación afecta la mantenibilidad.



Desarrollo Práctico

Propuesta de refactorización

Plantear una mejora concreta del diseño o del código:

- separación de responsabilidades;
- extracción de métodos;
- extracción de clases;
- reutilización de funciones comunes;
- centralización de validaciones o reglas compartidas;
- reorganización de componentes.

Comparación antes y después

Presentar una comparación clara del estado actual y la propuesta mejorada:

- antes: problema detectado;
- después: solución refactorizada.

RÚBRICA PARA CALIFICACIÓN INDIVIDUAL (EXPOSICIÓN)



CATEGORÍA	PUNTAJE				MÁXIMO
	5	4-3	2-1	0	
Organización y Estructura de la Presentación	La presentación sigue una estructura lógica y es fácil de seguir.	La estructura es buena pero puede ser mejorada en algunas áreas.	La presentación es confusa o carece de organización.	La presentación no sigue una estructura coherente.	5
Dominio del Tema y Capacidad de Responder Preguntas (Vale el doble)	Demuestra un sólido dominio del tema y responde a todas las preguntas con confianza y precisión.	Muestra un buen conocimiento del tema, aunque con algunas dudas.	Tiene dificultades para responder preguntas, mostrando un dominio limitado del tema.	No pudo responder satisfactoriamente a las preguntas.	10
Calidad Visual y Profesionalismo de la Presentación			La presentación es visualmente atractiva, profesional y sin errores ortográficos o gramaticales.	La presentación es desordenada y carece de profesionalismo.	2
Participación		Entre 81% y 100%	Entre 61% y 41%	<=40%	3
TOTAL					20

Conclusiones

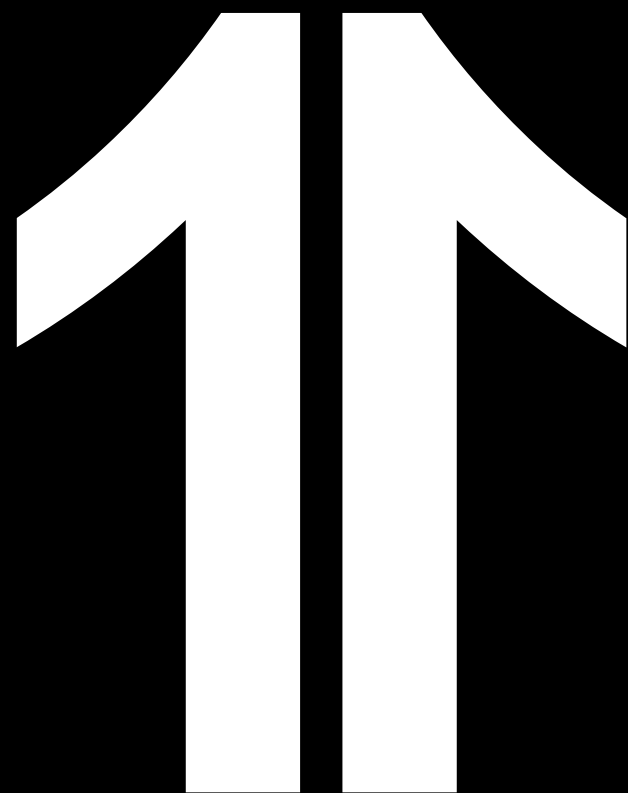


- La refactorización permite mejorar la calidad interna del software sin cambiar su funcionamiento, facilitando su comprensión, mantenimiento y evolución.
- El Principio de Única Responsabilidad (SRP) ayuda a diseñar clases y módulos más claros, organizados y fáciles de modificar.
- El principio DRY evita la duplicidad de código, reduce errores y promueve soluciones más reutilizables y eficientes.
- Aplicar estas buenas prácticas contribuye a desarrollar software más limpio, escalable y profesional.
- En el ejercicio profesional, refactorizar oportunamente permite ahorrar tiempo, reducir costos de mantenimiento y mejorar el trabajo en equipo.



!!! MUCHAS GRACIAS!!!
por su atención...

Ing. Edgard Vilchez Valdez



UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**